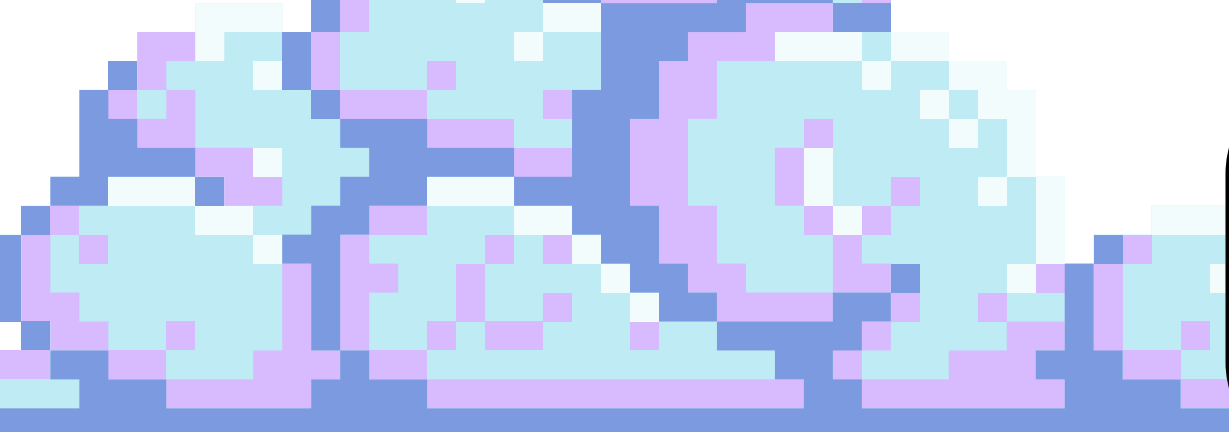


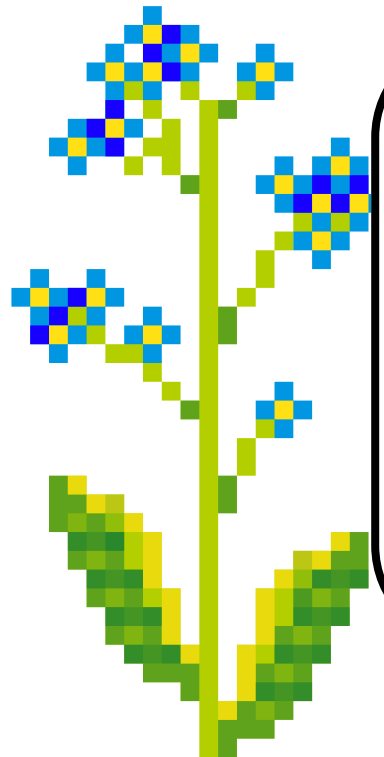


Eternal Clash 2

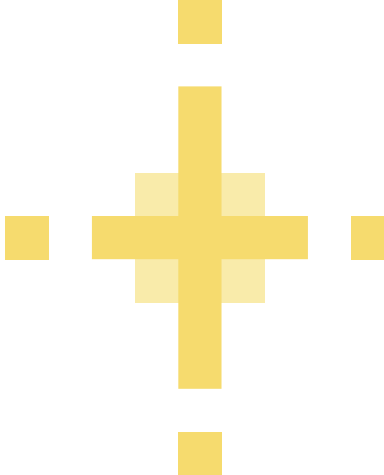


ปัญหาของโครงการ กลุ่มเป้าหมาย

ปัญหา : เกมวางแผนบนเว็บส่วนใหญ่มักมีระบบที่ซับซ้อนเกินไป หรือใช้เวลาเล่นนานเกินไปจนไม่เหมาะกับเวลาว่างสั้นๆ

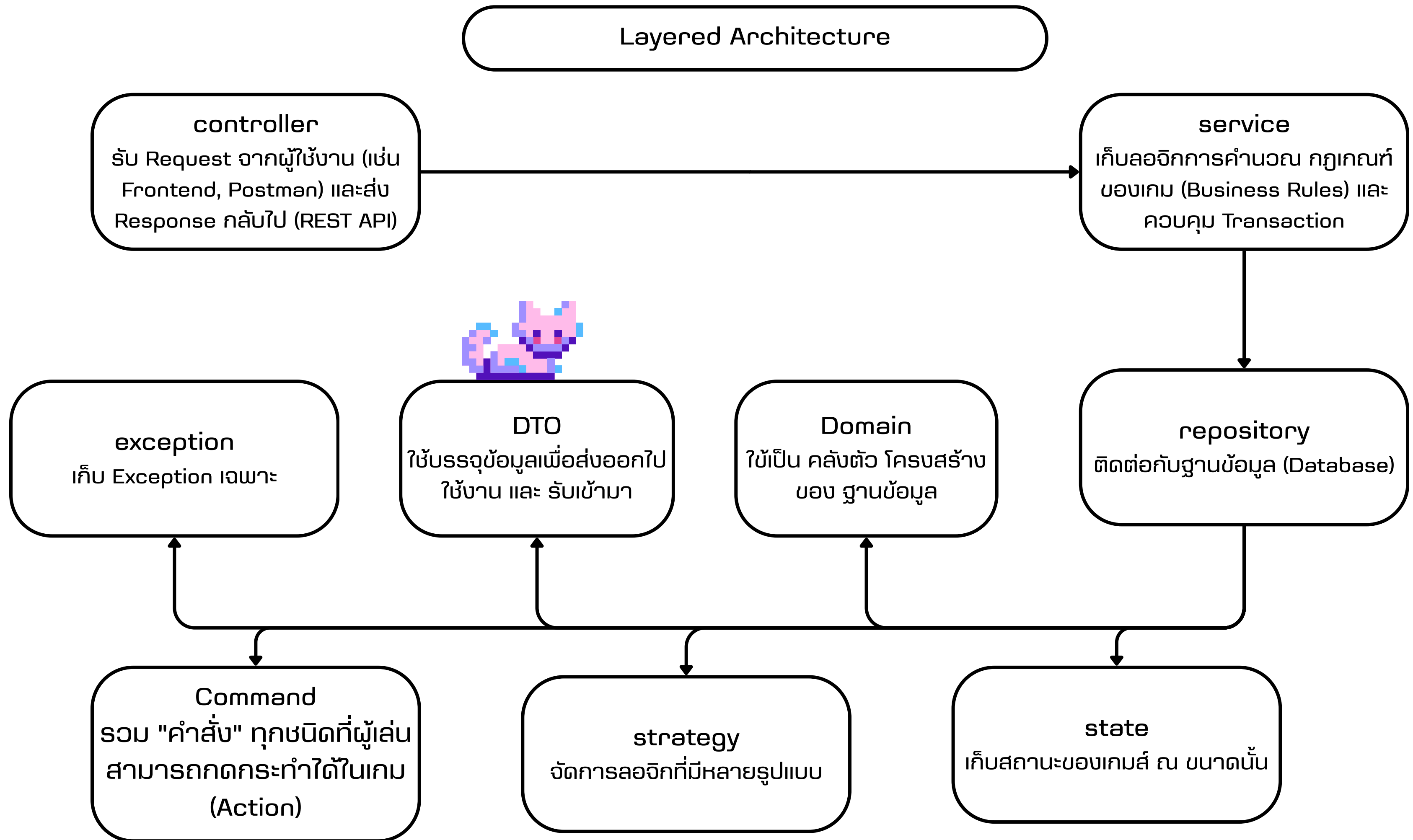


กลุ่มเป้าหมาย : ผู้เล่นที่ชื่นชอบเกมแนววางแผน (Strategy) แบบ Turn-based ที่ต้องการเกมจบไว เข้าใจง่าย และเล่นกับเพื่อนได้ทันที



ขอบเขต

- Tech Stack: React (Vite) + Java Spring Boot + PostgreSQL (Supabase)
- รองรับผู้เล่นหลายคนแบบ Real-time ผ่านการแชร์ห้อง (Room Code)
- ระบบบุณพล (Marshal) ที่มีสกิลและกลยุทธ์ต่างกัน 7 ตัวละคร
- ระบบ Event ภัยพิบัติแบบสุ่ม และระบบต่อสู้คำนวณแพะชนะอัตโนมัติ



- S** เราแยก BattleService (ลอจิกต่อสู้) ออกจาก TurnService (ลอจิกรับ
เทิร์น) เพราะถ้ามัดรวมกันไฟล์จะยาวเป็นพันบรรทัด
- O** MarshalAbilityContext.java ดึงความสามารถขุนพลผ่าน Strategy Pattern และ Registry Map
หากมีขุนพลใหม่ แค่เพิ่มคลาสเข้าไปใน Map ระบบก็พร้อมใช้ (Plug & Play) โดยไม่ต้องแก้โค้ด if-else
เดิมเลย
- L** KongmingCombatStrategy.java สืบทอดจาก StandardCombatStrategy และเพิ่มความสามารถ
เรื่องโอกาสรอดโดยยังใช้แทนคลาสพื้นฐานผ่าน CombatStrategy ได้ตราบใดที่ผู้เรียกใช้ยอมรับ
พฤติกรรมการรอดแบบสุ่มนี้
- I** PlayerActionCommand.java ทำให้คำสั่งเกมใช้ interface ร่วมกันได้ แต่มีจุดที่ควรปรับ: คำสั่งผลิต
อาหารและเกณฑ์ทหารต้องมี getArmy() ทั้งที่ไม่มีกองทัพและจึงคืนค่า null
ProduceFoodCommand.java อาจแยก getArmy() ไปไว้ใน interface ย่อยสำหรับคำสั่งส่งทัพ
- D** รับ GameRepository, PlayerRepository และ repository อื่นผ่าน constructor ที่ Lombok สร้างให้
จาก @RequiredArgsConstructor จึงพึ่งพา interface ของ repository แทนการสร้างคลาสเชื่อมฐาน
ข้อมูลขึ้นมาเอง



Design Patterns

State Pattern

เนื่องจากตัวเกมส์มีหลาย
เฟส(State)ในการทำงาน ซึ่งการ
ทำงานของ แต่ละ เฟสมีการกระทำ
ที่ต่างกัน ผู้จัดทำจึงนำ State
Pattern มาปรับใช้ในส่วนนี้

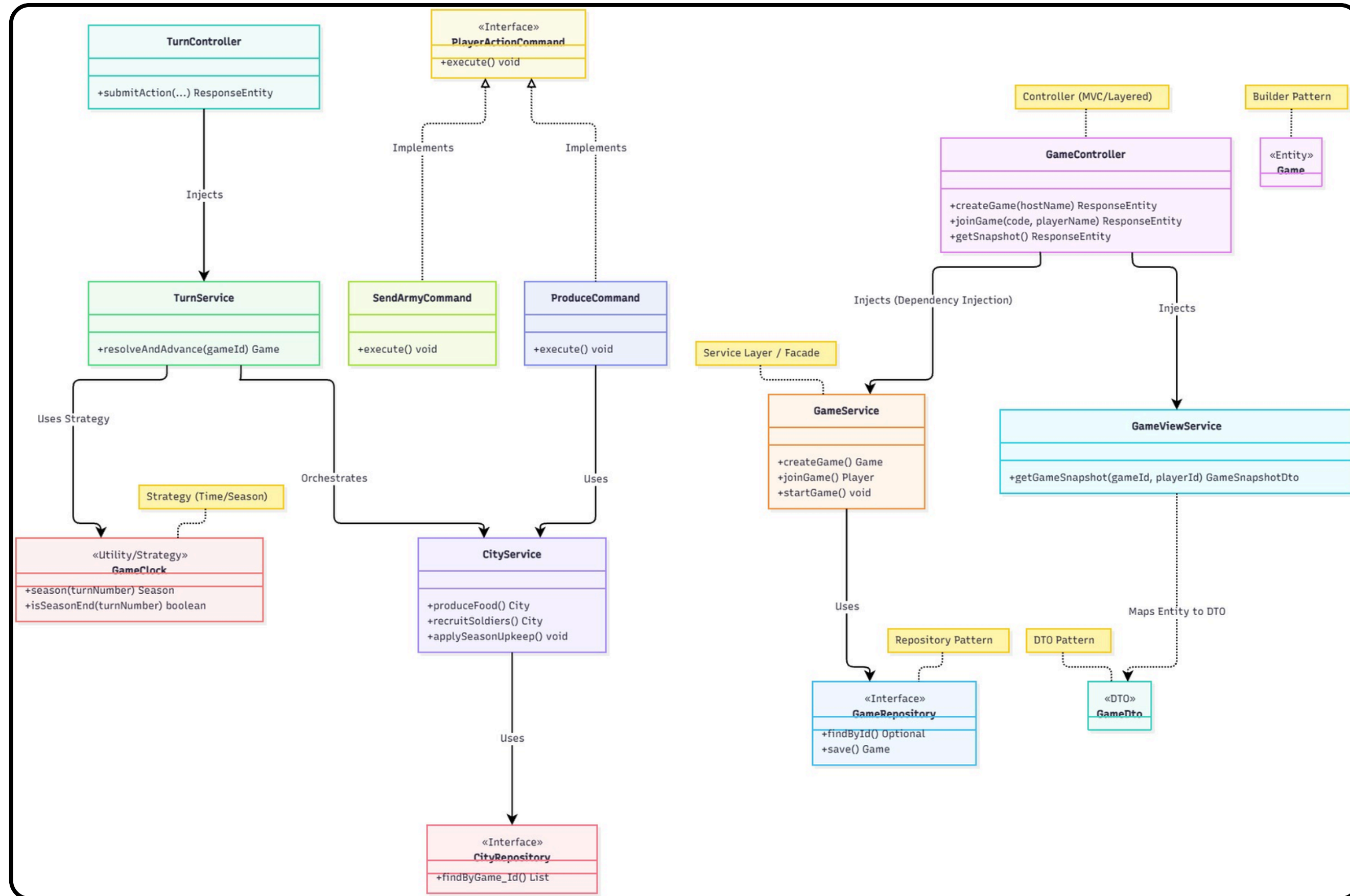
Command Pattern

Action แต่ละตัวต้องมีรูปแบบที่
ต่างกันและเมื่อในอนาคตจะมีการ
เพิ่ม อีกทั้ง Action จะทำงานจะ
ต้องรอ ให้รวบรวม Action ของ
ผู้เล่นทุกคนก่อน

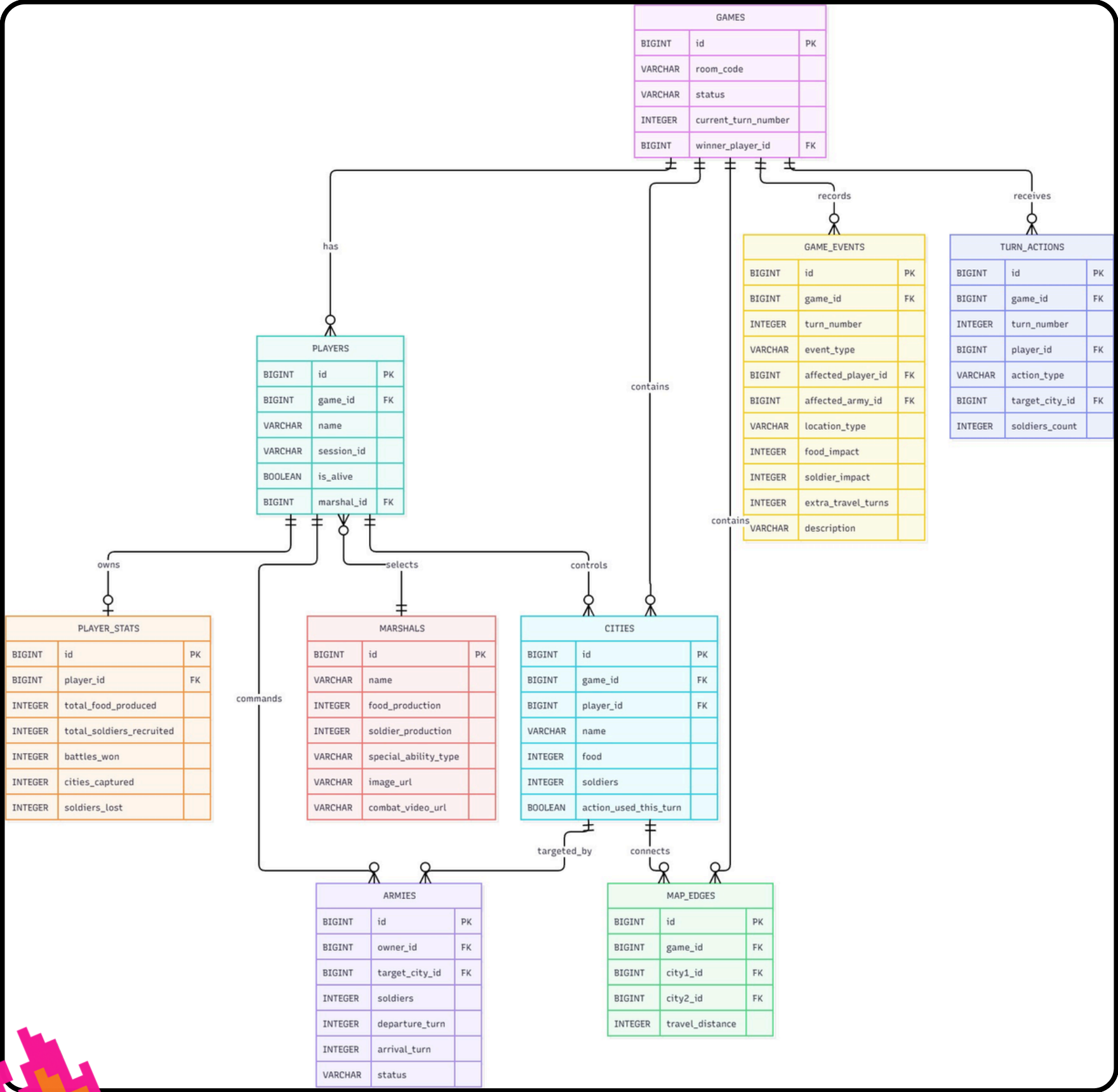
Strategy Pattern

เพราะ การคำนวณไม่ว่าจะทั้ง
การคำนวณความสูญเสีย หรือ
คำนวณ ผ่าน คลาส จอมพล
ทำให้การออกแบบ โดยเรียกใช้
Strategy Context จัดการในการ
เลือก Strategy ที่เหมาะสมกับ
งานนั้น

Class Diagram



ER Diagram



ความสัมพันธ์

games → players

players ↔ player_stats

players → cities

cities ↔ cities

players → armies

players → marshal_candidates

marshals → players

marshals → marshal_candidates

games → turn_actions

games → game_events

games → battles

ประเภท

One-to-Many

One-to-One

One-to-Many

Many-to-Many

One-to-Many

One-to-Many

One-to-Many

One-to-Many

One-to-Many

One-to-Many

One-to-Many

คำอธิบาย

1 เกมมีหลายผู้เล่น

1 ผู้เล่นมี 1 สถิติ (แยกเพื่อลดภาระ Query)

1 ผู้เล่นเป็นเจ้าของได้หลายเมือง (NULL = เมืองร้าง)

เมืองเชื่อมต่อกันไปมาได้หลายทาง (ทำผ่านตารางกลาง map_edges)

1 ผู้เล่นส่งได้หลายกองทัพ

1 ผู้เล่นได้รับ 3 ตัวเลือกจอมพล

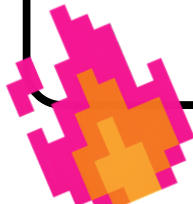
1 จอมพลถูกเลือกโดยหลายผู้เล่น (คนละเกม)

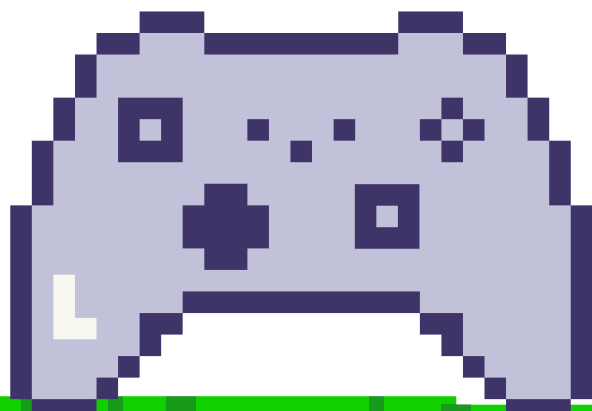
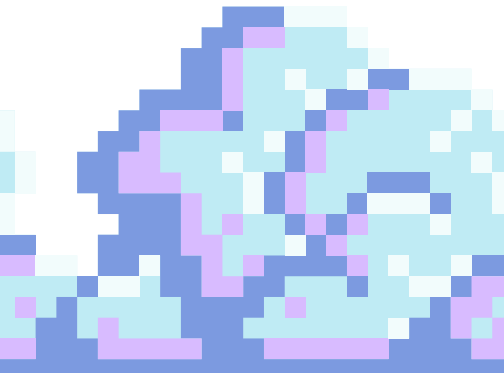
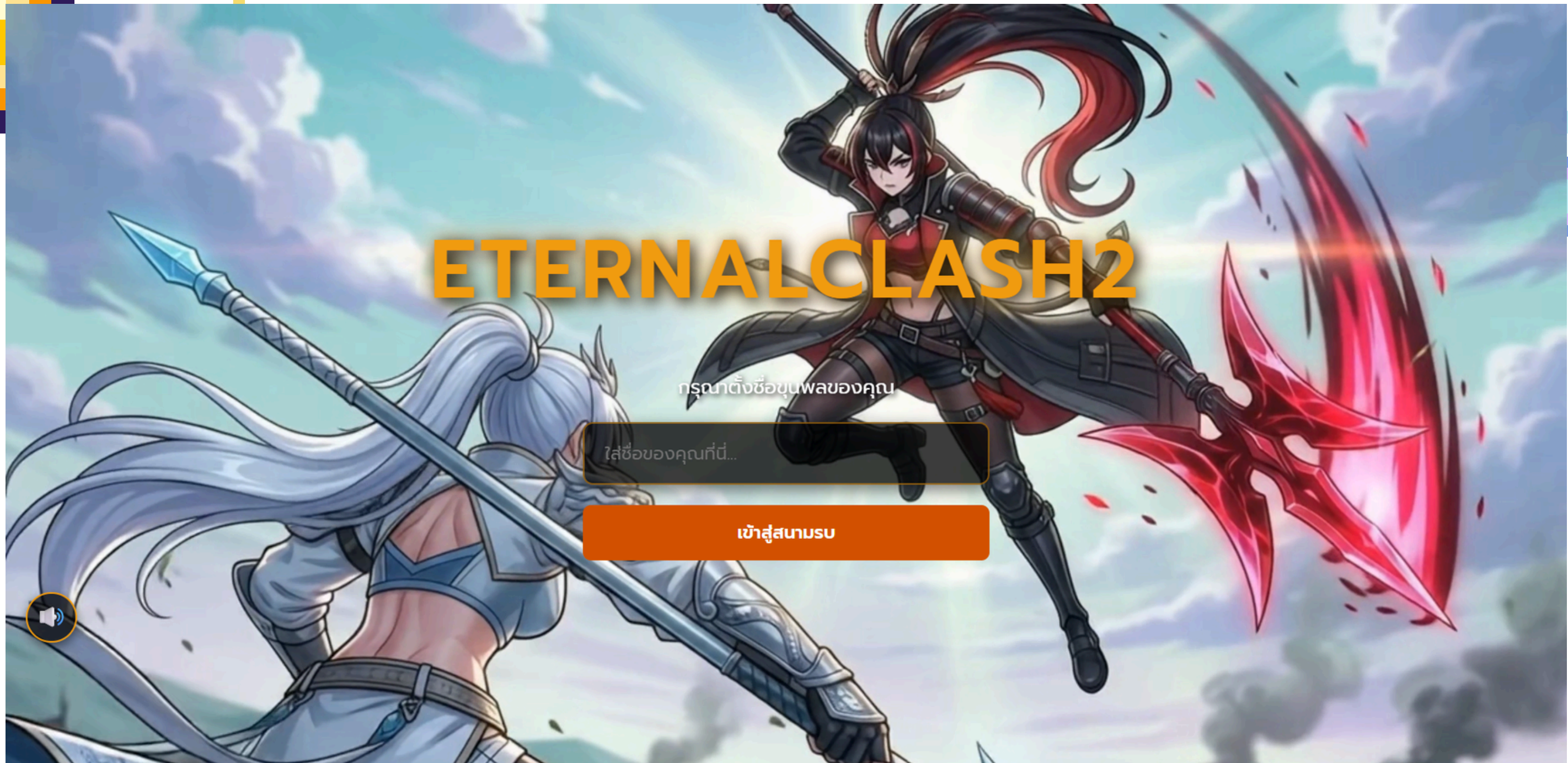
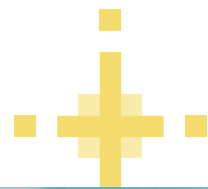
1 จอมพลเป็นตัวเลือกของหลายผู้เล่น

1 เกมมีหลาย Action log

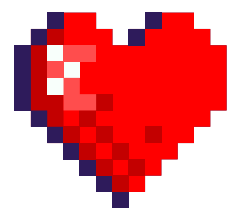
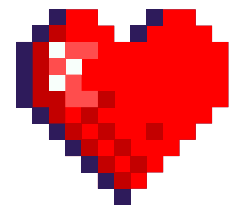
1 เกมมีหลาย Event

1 เกมมีหลายการต่อสู้





Demo



ขอบคุณค่ะ

